

# 編譯並執行 .NET 應用程式

- **4-1: 應用程式進入點**
  - Entry Point 進入點 ..... 4-1
  - 宣告實作及呼叫方法 ..... 4-2
- **4-2: 認識組件(Assembly) & Metadata**
  - 編譯與組件(Assembly) ..... 4-3
  - 中繼資料Metadata ..... 4-5
- **4-3: 應用程式如何執行**
  - 設定啟始專案 ..... 4-6
  - 在Visual Studio中執行 ..... 4-7
  - 在專案資料夾中執行 ..... 4-8

## Module 04

# Entry Point 進入點

- 進入點為程式(專案)執行時，第一個執行的方法。
- 一個專案只會有一個進入點。
- 下圖 `private static void Main(string[] args)` 即為進入點。

```
public class Program
{
    private static void Main(string[] args)
    {
        //程式碼
    }
}
```

# 宣告實作及呼叫方法

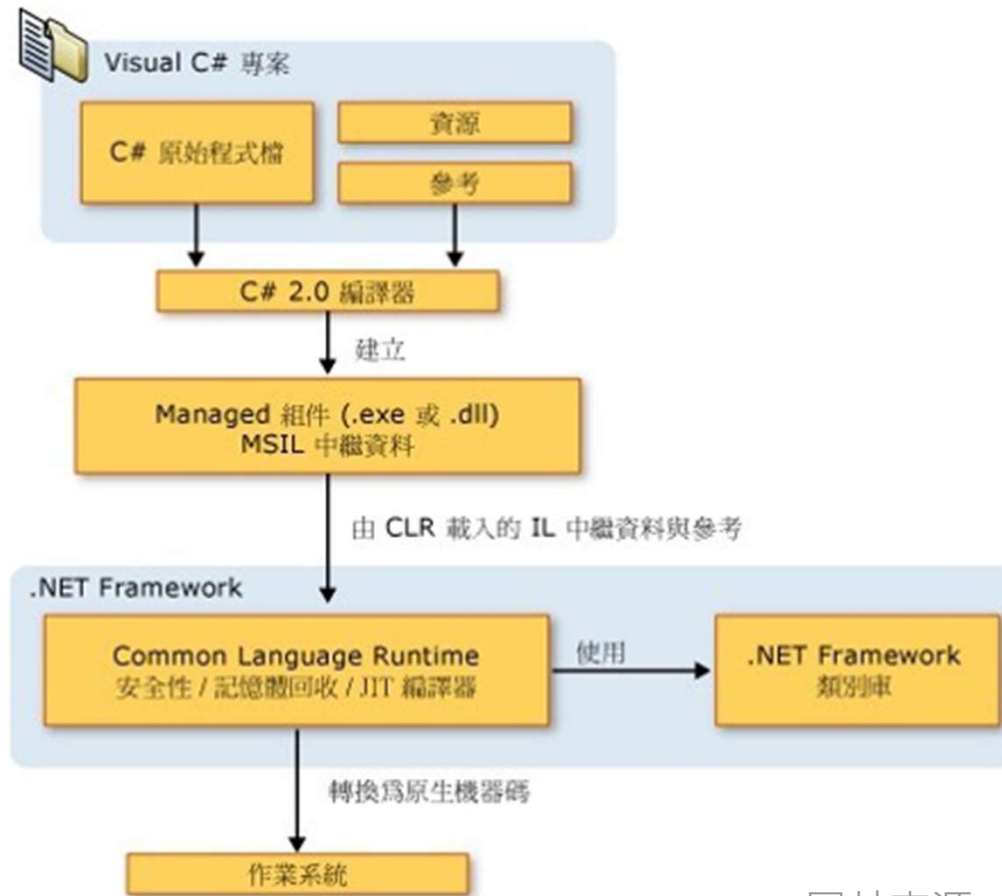
- 物件必須建立實作後才能使用(instance屬性)。
  - `MyClass myclass = new MyClass();`
  - `myclass.MyVoid();`
- 若物件為靜態屬性(static)則不須建立new即可使用。
  - `Console.WriteLine(myclass.str2);`
- “.”為Scope Resolution，用於選取物件的成員。
  - `Console.WriteLine("Scope Resolution");`

## 編譯與組件(Assembly)-1

- 當完成程式原始檔(.cs)時，必須透過編譯器進行編譯，並組件封裝成.exe或.dll檔才可執行。
- 組件完成的檔案儲存的是中繼資料，程式碼轉換為 **Microsoft Intermediate Language (MSIL)** 成為檔案的一部分。
- 在程式執行時，**.NET** 會透過 **JIT 編譯器 (Just-In-Time Compiler)**，將**MSIL**即時編譯為特定**CPU**架構的機器語言，由系統載入執行。

# 編譯與組件(Assembly)-2

- C#的編譯與執行過程。



# 中繼資料 Metadata

- 中繼資料描述程式碼中的每一個型別和成員。
- 中繼資料儲存下列資訊：

- 組件的說明

- 識別 (名稱、版本、文化特性、公開金鑰)
- 匯出的型別
- 這個組件所依存的其他組件
- 執行所需要的安全性權限

- 型別的說明

- 名稱、可視性、基底類別和實作的介面
- 成員 (方法、欄位、屬性、事件、巢狀型別)

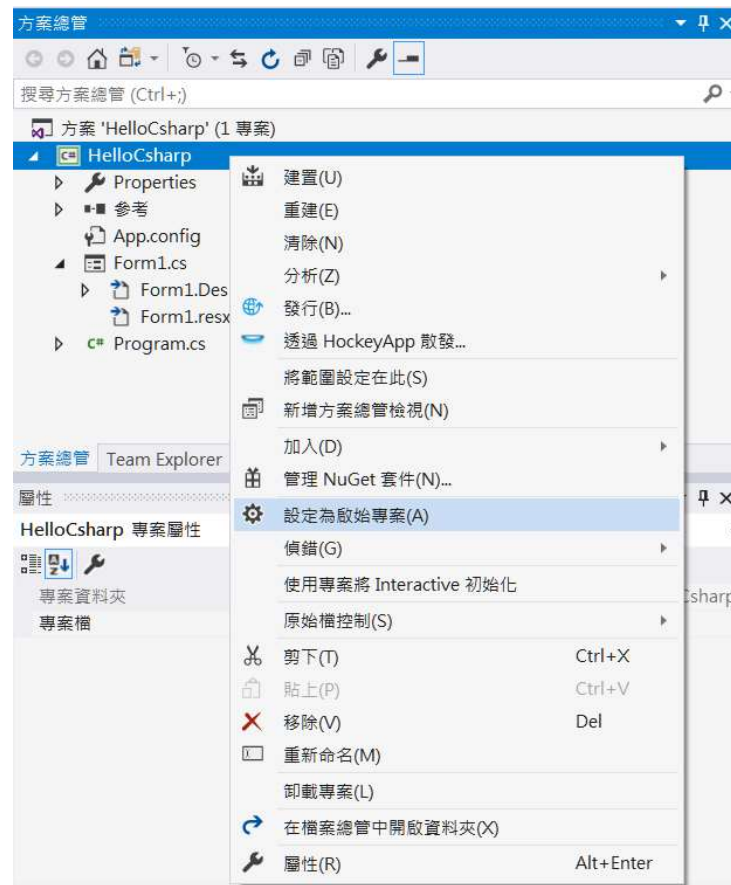
- 屬性

- 修改型別和成員的額外描述項目

參考資料：<https://docs.microsoft.com/zh-tw/dotnet/standard/metadata-and-self-describing-components>

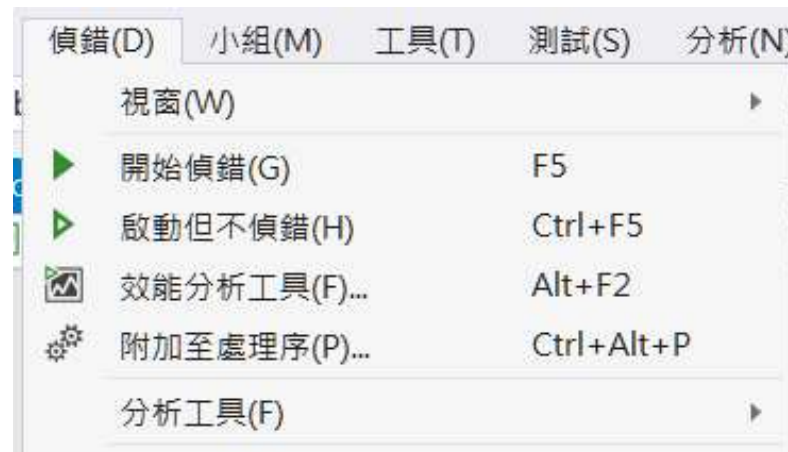
# 設定啟始專案

- 在[方案總管]選定欲設定的專案點右鍵，按下[設定為啟始專案]。



# 在Visual Studio中執行

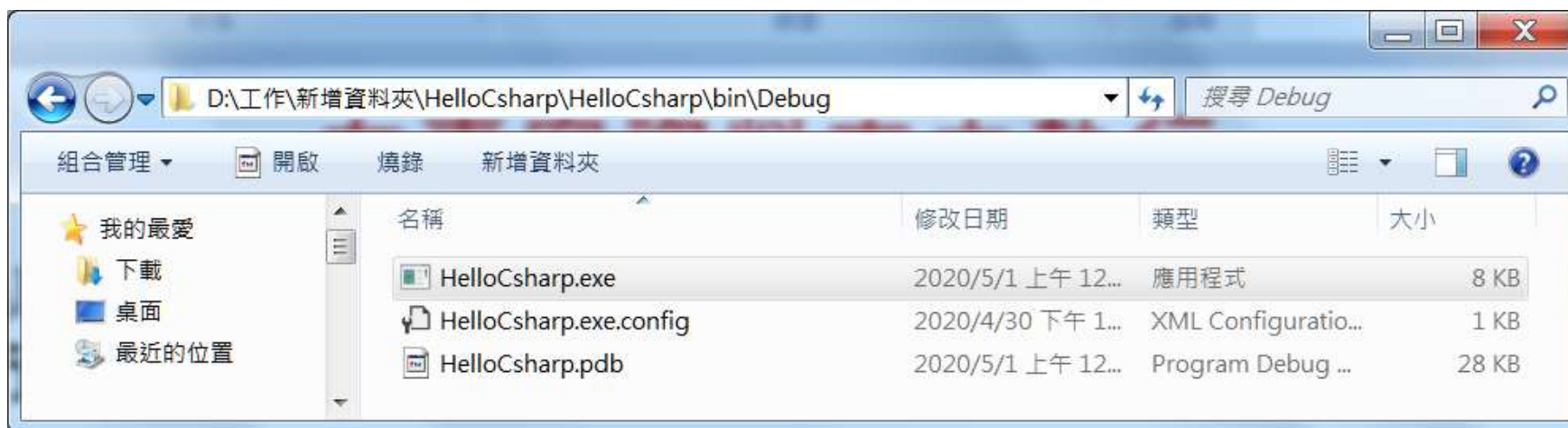
- 按下工具列上的  或鍵盤上[F5]鍵可以偵錯模式執行。
- 按下鍵盤上[Ctrl] + [F5]鍵可以不偵錯模式直接執行。
- 或可點選功能列[偵錯]，透過下拉選單可選擇偵錯或不偵錯執行。





## 在專案資料夾中執行

- 可於 \專案根目錄\bin\Debug 下選取執行[專案名稱.exe]，可執行欲執行的專案應用程式。



- 若須將應用程式製作成安裝檔，須另外安裝Microsoft Visual Studio Installer Projects工具套件。